SR Parser Overview

CMPS 450, Dr. Maida

September 20, 2026

Let's begin by making some terms explicit. An LR parser is a bottom up parser. You may remember that L stands for scanning the input from left-to-right and R stands for rightmost derivation. The most common parser to implement the bottom-up algorithm is the shift-reduce (SR) parser. So, for this class LR parser and SR parser are synonyms.

1 The parts of an SR parser

Figure 1 shows a diagram for an SR parser taken from the textbook. It has three parts: an input queue of tokens, a parse stack that is internal to the parser, and a parse table that is internal to the parser.

input queue The input queue is labeled "input" in the figure. This is a normal queue whose elements are tokens with a special end-of-input symbol, which is a dollar sign. In figure 1, we are midway through the parse so the front of the queue is token a_i. **The front of the queue is on the left.**

Parsing table An SR parser is a table driven parser. That is it uses a static table to tell it what to do. The table is constructed by an algorithm which analyses the grammar. For this class, we assume that the table is given to us. The table has two parts: an action subtable and a goto subtable (see Fig. 2).

Parse stack There is another data structure within the parser, called the parse stack, which is dynamically modified during the parsing process. **The top of the stack is on the right.**

2 Deciding what action to take

The input is a token sequence. The parser processes the input tokens 1-by-1. When the parser is processing an input token, a_i, it can decide to take one of four actions. The actions are: shift, reduce, accept, and error (ungrammatical). Making the decision is a two-step

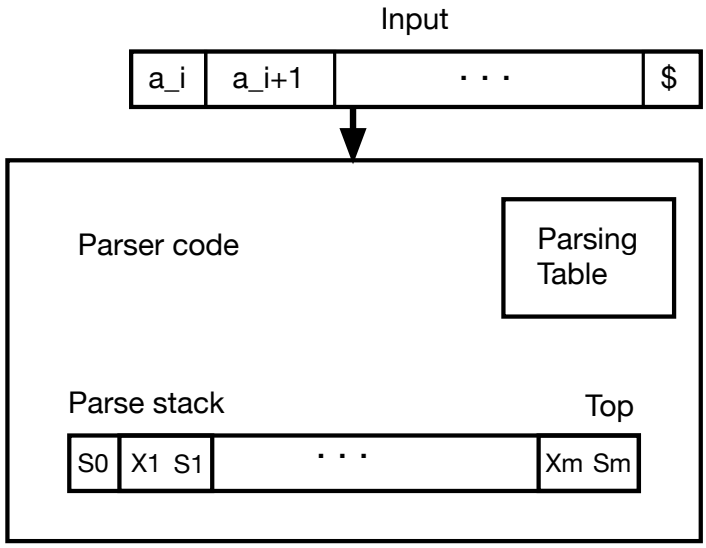


Figure 1: SR parser.

process. First, the parser looks at the current input token and also the information at the top of the parse stack. Second, with this information, it uses the table to tell it what to do.

We are going to walk through this process at a conceptual level. Our original grammar is repeated below.

E → E + T | T
T → T * F | F
F → (E) | id

We now expand the grammar at all of the vertical bar choice points, and eliminate the vertical bars. That is, whenever there is a vertical bar, we break the rule into two rules. This is shown below. Also **each grammar rule now has a rule number** which will be used in the algorithm.

```
1. E --> E + T
2. E --> T
3. T --> T * F
4. T --> F
5. F --> ( E )
6. F --> id
```

Now let's think about the information in the parse stack shown in Figure 1. Except for the item at the bottom of the stack (leftmost), **each stack entry has two pieces of**

	Action						Goto		
state	id	+	*	(	)	$	E	T	F
0	S5			S4			1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

Figure 2: The parse table has two subtables: action and goto.

information namely, an X_m and an S_m. An S symbol denotes a state that can be looked up in the parse table and an X symbol denotes a symbol in the grammar.

2.1 Initial configuration of the parser

"Configuration" is a fancy word for state. We want to talk about the state of the parsing process, but we already used the word for a state in a parsing table. Formally, the initial configuration of the parser is given below. It describes the configuration of the parser at the beginning of the parsing process. It consists of the comma-separated pair:

$$(S_0, a_1 \dots a_n\$) \tag{1}$$

In the above, the comma symbol separates the two parts of the pair. **Make sure you can find the comma.** The left part describes the **initial parse stack**. It has only one entry, which is the initial state, named S_0. The right part describes **the initial token queue**. It consists of n tokens denoted a_i and the dollar sign symbol to signify the end of the input. **The input is n tokens long**, so this includes all of the input.

2.2 Using the parse table

The parse table controls the parser actions. The table is different for each grammar.

The parse table for the grammar that we are using is in Figure 2. It has two parts an Action subtable and a Goto subtable. Even for this simple grammar the table is too complex to build by hand. There is an algorithm that analyzes the grammar and generates

the table. We will not study that algorithm in this class. For this class, you only have to know how to use a parse table that is given to you.

For this tutorial, we only use the Action subtable and only do the first step of the parsing process. To decide what action to take, you look up an action in the action table. You look up an entry in the action table by finding the state in the leftmost column and the token you are looking at in the top row. Let's assume that the input is:

```
id + id + id
```

Then the first token in the queue is `id` which is denoted a_1. Also, remember that in the initial configuration of the parser, the state at the top of the parse stack is S_0.[1] If we look in the parsing table to find the row for state "0" and the column for "id", we find the value "S5". **The "S" in the table stands for the shift action** and the "5" is the new state id we are moving into (Previously, it was "0"). So the table is telling us to perform a shift action. To perform the shift action, we move the token from the front of the queue, and we combine it with 5 to create a new pair. We push this pair onto the parse stack. The new configuration of the parser is given below.

$$(S_0 \;\; id\, S5, a_2 \ldots a_n\$) \tag{2}$$

In the above, we have pushed the pair $id\, S5$ onto the parse stack, and we have also removed the first token, $a_i = a_1$, from the input queue. So, **there were two changes**: 1) we pushed something onto the stack; and, 2) we removed something from the queue.

Take another look at the action table. Note that **there are four kinds of entries**. If an entry begins with an "S", it stands for the shift action (which we just used). If an entry begins with an "R", it stands for the reduce action. If an entry holds the word "accept", the parsing is finished and it was successful. Notice that **it only makes sense to have an "accept" in the dollar sign column** because the parse needs to look at the whole input queue to know if the parse is successful. Finally, **empty entries indicate that the input was ungrammatical**. SR parsers are known for their ability to detect errors very quickly.

3 Summary

This tutorial described the components of an SR parser and gave you a taste of how the SR algorithm is driven by a parsing table. The algorithm is complicated, so we will go over it in another tutorial.

[1] The stack is empty, except for S_0, so it's at the top.